Artificial Intelligence
Final Project Report

Object Detection with YOLO
Dataset: PASCAL 2007

Author:

Hugo Bueno Buhigas - LS2402202

Artificial Intelligence - Final Project

LS2402202 - Hugo Bueno Buhigas
Object Detection - PASCAL 2007

Contents

1 Introduction **1**
1.1 PASCAL 2007 1

2 Experiment **2**
2.1 Dataset reorganization 2
2.2 Training and Validation using YOLO architecture 2

3 Results **3**
3.1 Confusion Matrix 3
3.2 Others 3
3.2.1 F1 Curve 3
3.2.2 PR Curve 4
3.2.3 Training results 4

1 Introduction

The present project covers an AI object detection model that works on the chosen PASCAL 2007 dataset.

Object detection is a fundamental task in computer vision, focusing on identifying and localizing objects of interest within an image. Unlike image classification, which assigns a single label to an entire image, object detection not only classifies objects but also predicts their locations using bounding boxes. Applications of object detection are vast, ranging from autonomous vehicles and video surveillance to healthcare and augmented reality. Over the years, advances in deep learning have significantly improved the performance and accuracy of object detection models, with architectures like YOLO, SSD, and Faster R-CNN leading the field.

1.1 PASCAL 2007

The PASCAL VOC (Visual Object Classes) 2007 dataset [1] is a widely recognized benchmark for object detection. It contains 9,963 images across 20 object categories, with over 24,000 annotated objects in total. These images span diverse scenarios, offering challenges like object occlusion, varying illumination, and scale changes. PASCAL VOC 2007 includes pre-defined train, validation, and test splits, making it a valuable resource for training and evaluating object detection algorithms. Its rich annotations and challenging nature have played a significant role in advancing object detection research.

This is its structure

- Annotations.xml files with all the information about each image (author, label, date, etc.).
- ImageSets: contains the .txt with the classification of each image in the subdirectory *Main*.
- JPEGImages: all .jpg images together.
- Segmentation Class
- Segmentation Object

2 Experiment

A description of the methodology used and involved files

2.1 Dataset reorganization

The structure of the dataset is rather inconvenient for dealing with the model for the following reasons: first, the training and validation images are all together in the same directory, knowing which is which according to the lists provided by *train.txt* and *val.txt* files in another directory; moreover, the correspondent labels of the images are mixed with much more data in the .xml files in *Annotations*.

The model struggled to find the images, most probably because only the number of the images was given and not the correspondent path. Furthermore, the model could find no labels to compare the quality of its execution. That is why I decided to reorganize the dataset in order to make it easier for the processing. This reorganization consists of three main steps, each of which has an associated program.

1. **Separation of images**: create two new directories (*train and val*) under the same root directory (VOC2007), separating the images in *JPEGImages* directory according to their .txt distribution.

2. **Extraction of labels**: get only the useful data from the associated .xml files (id and dimensions of the bounding box) and create .txt files in a directory under the same directory as the associated train/val images.

3. **Separation of images**: verify that the pre-processing was successful and that all images have their correspondent labels within the same directory.

Finally, *class.yaml* file was generated to specify the path to these directories and the number and names of the associated classes. This is a mandatory file for the architecture that is about to be introduced.

2.2 Training and Validation using YOLO architecture

For the model training and validation, the YOLO architecture [2] has been used (concretely, version 8 for small to medium-sized datasets). This is an open-source technique widely used in AI applications due to its efficiency and robustness when dealing with this area of expertise.

By simply specifying the operation parameters, YOLO trains and validates the images following the path provided by the .yaml file. It makes predictions based on intersections between bounding boxes, which can have more than one element to be detected per image. Later, it generates several results, including the weights of the trained model, the confusion matrix and several curves, all of which are found in runs/detect/trainX, being X the number associated to the particular execution.

Due to the fact that my computer is relatively new, I did not install yet the CUDA capabilities to use my GPU. Thus, my execution was ran by my CPU, taking a great deal of time for this relatively small dataset (around 3 hours). That is why two experiments have been carried out: a first one with 416 image size and 10 epochs, and a later one with 640 image size and 15 epochs (which hypothetically trades execution time for a higher precision).

3 Results

Metrics and graphs

3.1 Confusion Matrix

The normalized confusion matrix highlights the model's strengths and weaknesses across the 20 object classes. Classes like aeroplane, cat, and horse show high true positive rates, indicating strong performance likely due to sufficient and diverse training samples. In contrast, classes like chair and pottedplant suffer from notable misclassifications, possibly caused by visual similarities with other objects or complex backgrounds.

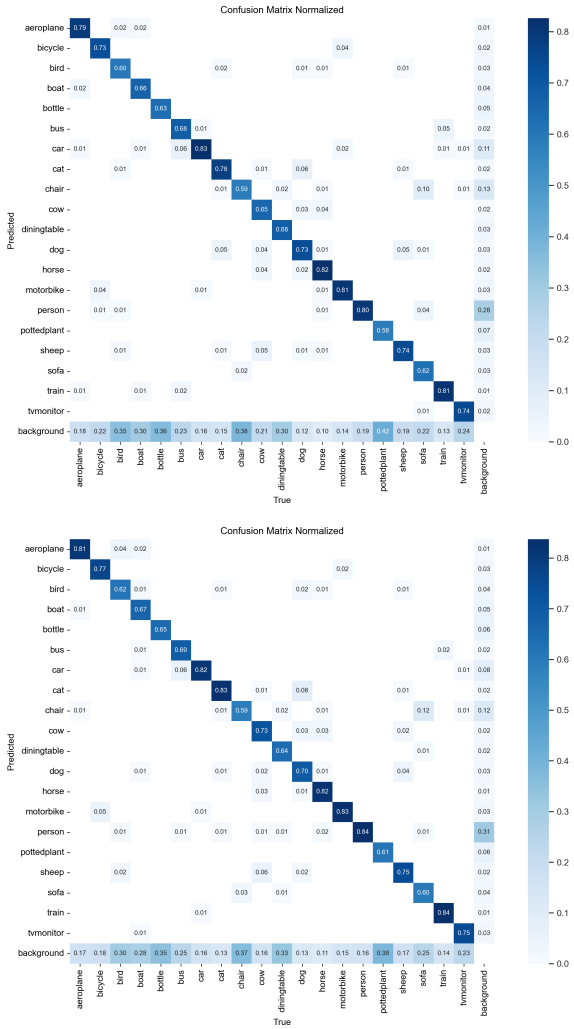
Inter-class confusion is particularly evident for visually similar categories (e.g., sofa vs. chair) and some objects blending with the "background" class. This suggests the need for improved data augmentation, class-specific oversampling, or enhancements in feature extraction through architectural adjustments.

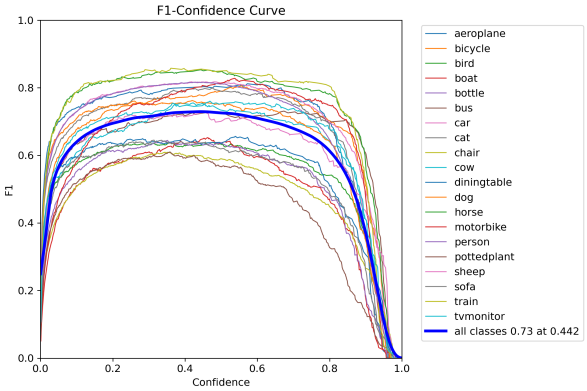
Besides, the comparison between the 416 matrix (first) and the 640 (second) proves the image size and number of epochs do have a positive trade-off in the model's precision, observing higher correlation in almost all the true positives.Overall, the model demonstrates solid performance but struggles with a few challenging categories, signaling opportunities for refinement, particularly in handling difficult or underrepresented classes.

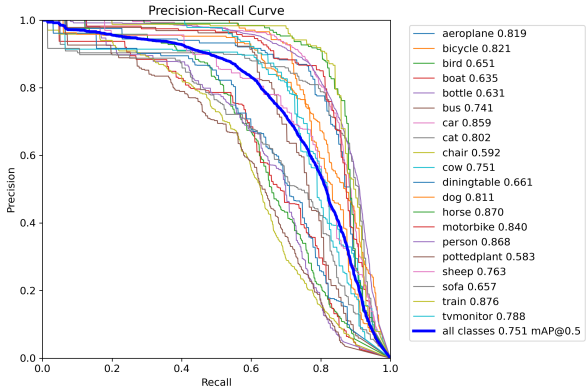
3.2 Others

3.2.1 F1 Curve

The F1-confidence curve reveals how well the model balances precision and recall across various confidence thresholds. The peak F1 score of 0.73 at 0.442 confidence demonstrates reasonable overall performance, but some classes show inconsistent trends. For instance, classes like aeroplane and person achieve high F1 scores, while others like pottedplant and chair lag behind, hinting at challenges with more ambiguous or less frequent objects. Adjusting the confidence threshold or rebalancing the dataset could help address these gaps and enhance the overall balance between precision and recall.





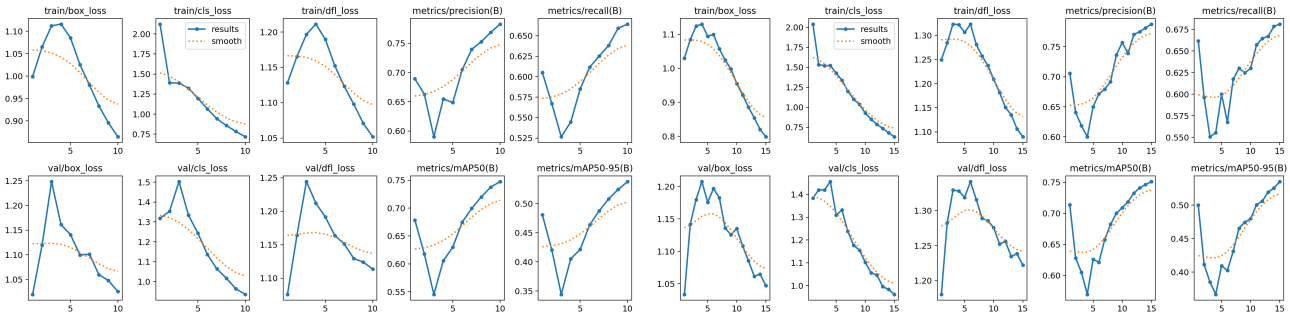


3.2.2 PR Curve

The precision-recall curve highlights variability in class-level performance. Classes like horse and train achieve high precision and recall, indicating reliable detection across scenarios, while bottle and pottedplant show steeper precision drop-offs at higher recall values, reflecting difficulties in consistently distinguishing these objects. The overall mAP@0.5 of 0.751 is a strong baseline, though narrowing the performance gap between classes remains critical for improvement. Focused attention on underperforming classes could boost generalization.

3.2.3 Training results

The training results (left for image size 416, right for 640) show consistent improvements across metrics like precision, recall, and mAP throughout the epochs. The decreasing box, classification, and DFL losses indicate steady optimization, with validation loss trends mirroring those of the training set, suggesting minimal overfitting. However, slower improvements in mAP50-95 compared to mAP50 hint at challenges with detecting objects at stricter IoU thresholds. Fine-tuning the learning rate, increasing the number of epochs, or improving anchor box configurations could further optimize performance.



References

[1] KAGGLE dataset https://www.kaggle.com/datasets/lottefontaine/voc-2007

[2] YOLO architecture https://docs.ultralytics.com/es/models/yolov8/